

# Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic\_netops\_validator\_v5 for deterministic end-to-end correctness checks; preregistration used shared\_initial\_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model\_calls\_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.

## I. INTRODUCTION

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study’s preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor

CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic\_netops\_validator\_v5), and the analysis executor (paired\_binary\_analysis\_v1).

A decisive preregistered design choice was shared\_initial\_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

## II. RELATED WORK

Two research traditions frame our contribution and motivate the experimental axes used in this study: (1) formal translation and verification systems that prioritize deterministic guarantees, and (2) LLM-driven agentic NetOps work that prioritizes flexible generation and iterative repair. Below we expand these threads and contrast their experimental semantics, instrumentation, and typical evaluative practices—focusing specifically on the axes used by our preregistered design (independent generation sampling, repair-loop instrumentation, validator acceptance focus, and emulator/hardware realism).

**Formal translation→verification.** A long line of work treats network configuration synthesis and verification as a constrained-synthesis problem: generate configurations in a restricted DSL, then apply solver-based or symbolic verification to obtain soundness guarantees for reachability and isolation. Seminal systems such as SymNet and follow-on "Don’t

Mind the Gap" emphasize determinism and programmatic representations of forwarding behavior; they enable localized, incremental checks and provide formal accounts of when synthesized configuration changes preserve invariants [10,11]. More recent programming-language and verification advances (e.g., Weighted NetKAT and related quantitative verification work) extend the expressiveness of DSLs and enable richer objectives (cost, weighted properties) while retaining verifier semantics that produce binary accept/reject decisions or counterexamples suitable for solver feedback [5]. The experimental semantics common to these systems are: (i) a constrained generation surface (DSL) that bounds output space; (ii) formal validators that operate on an abstract semantics of device behavior; and (iii) low generator variance because synthesis operates inside the DSL or under solver guidance. Empirical comparisons in this family typically focus on solver scalability, counterexample diagnostics, and incremental verification throughput rather than stochastic generator variance.

LLM-driven and agentic NetOps. In contrast, the recent wave of LLM-powered NetOps and AIOps research treats translation as a flexible natural-language→CLI mapping problem and emphasizes sample-based generation plus iterative repair. Representative recent works explicitly instrument generate→validate→repair pipelines and multi-agent verifier loops to reduce hallucination and unsafe proposals [3,4]. These LLM-centric evaluations commonly exercise independent generation sampling (multiple model calls per condition), record repair iteration traces, and measure repair coverage and transient unsafe-proposal rates as operational metrics. Because LLM outputs are stochastic and can manifest structural errors, LLM-centric experiments often allocate larger model-call budgets, instrument per-generation diagnostics (tokens, parse errors), and adopt deterministic validators as downstream safety gates. Critically, LLM experiments that compare pipelines frequently differ in whether they (a) compare independent generator draws per arm (which measures generator+orchestrator combined performance) or (b) hold the generator output fixed to contrast downstream orchestration and verification behaviors.

Experimental axes and their methodological consequences. Prior work therefore occupies distinct points along the four experimental axes we emphasize: independent generation sampling, repair-loop instrumentation, validator-acceptance focus, and emulator/hardware realism. Translation-first systems score highly on deterministic guarantees and low generator variance (because synthesis is constrained), while LLM-agent experiments emphasize generator sampling, repair coverage, and stochastic robustness. Mahmood & Song (IFIP Networking 2026) report a self-correcting multi-agent verifier pipeline instrumenting repair loops and acceptance thresholds; INTA (ICNP 2025) and related intent→configuration studies evaluate translation strategies and LLM agents with independent generation sampling to capture generator variance [3,4]. Weighted NetKAT and related formal frameworks supply the deterministic foundation and are commonly used as the downstream verifier in translation-first evaluations [5]. These verified works

are the direct methodological relatives of the present run.

Positioning of the present run. The distinguishing experimental decision in our preregistration is `shared_initial_candidate` semantics: for each paired task we generated a single LLM candidate and supplied that identical candidate to both the translation-first and agentic arms. Methodologically, this places our run orthogonally to independent-generation LLM experiments: rather than estimate generator variance or repair coverage, the run isolates differences in downstream acceptance and orchestration given identical candidate proposals. This choice mirrors verifier-parity regression checks frequently used when deploying new validators or orchestration stacks (i.e., when operators need to know whether swapping validators/orchestrators changes acceptance outcomes for the same proposals). Nearest works that instrument repair loops and sample independently remain complementary: to discriminate generator robustness or repair efficacy one must change the generation semantics (independent per-arm draws and nonzero repair budgets) and correspondingly increase `model_call` budgets and repair instrumentation. Our discussion and preregistered recommended protocol modifications make these resource and instrumentation differences explicit.

Artifact and evaluation practices across prior work. A practical contrast concerns archival and per-task artifactization. Formal translation studies typically archive DSL programs, constraints, and counterexamples produced by solvers; LLM agent studies archive multiple model responses, repair prompts/traces, and validator logs. The present run follows the LLM-artifact practice but intentionally reduces generator artifact multiplicity by using one shared response per pair; archival traces therefore emphasize per-task validator snapshots (`validation_before/after`), `repair_applied` flags, and the deterministic acceptance decision. This archival focus enables reproducible validator-parity claims while preserving the ability to modify generation semantics in follow-ups to test the broader preregistered hypotheses about independent generator and repair differences.

### III. METHODOLOGY

This section expands the preregistered and executed methodology with concrete, artifact-grounded implementation and validation details. All referenced artifacts and parameter values are drawn from the archived execution and analysis bundle (paths cited where informative).

Preregistration and frozen contract. The study design was frozen in preregistration CAND-2026-001-prereg-v1. Key preregistered parameters used without post-hoc change: `planned_episode_count` = 240 (120 paired tasks), `model_scope` = {"gpt-5-mini"}, `execution_adapter` = `hosted_netops_gvr_v1`, `generation_semantics` = `shared_initial_candidate`, `maximum_model_calls` = 240, and `validator` = `deterministic_netops_validator_v5` (`validator_version` = 5.0). The preregistered analysis executor was `paired_binary_analysis_v1` with `bootstrap_resamples` =

10000 and `bootstrap_seed = 1729`. The preregistration SHA recorded in the execution manifest is provided for audit.

Task generation and difficulty metadata. Confirmatory tasks were produced deterministically by `deterministic_netops_task_generator_v5`. Each manifest entry (`execution/task_manifest.jsonl`) contains (a) the human-readable intent, (b) canonical reference answer (expected sequence of configuration commands), (c) initial and expected final device states, (d) a `required_sequence` listing required field-level operations, and (e) difficulty metadata fields used for stratification: `changed_interface_count`, `dependency_rule_count`, `level`  $\in$  {medium, hard, very\_hard}, `required_command_count`, and `transient_rule_count`. Representative manifest entries (e.g., `task-000001`) are archived; these entries are the ground truth used by the deterministic validator’s normalization and required-command checks.

Generation orchestration and shared\_initial\_candidate implementation. The orchestration implemented `shared_initial_candidate` as follows: for each paired task the adapter invoked a single LLM generation stage (stage key `"shared_initial_generation"`) and persisted the textual candidate. The single stored candidate was then supplied verbatim (no per-arm regeneration) to both the translation-first and agentic downstream arms for syntactic/semantic processing and deterministic validation. Execution accounting files record `model_calls_used = 120` (one LLM call per task pair) and provider audit metadata (`analysis/deterministic_reconciliation.json`) shows `hosted_model_call_event_count = 120`, `cache_miss_count = 120`, and `cross_condition_cache_key_reuse_observed = false`, indicating each pair used an isolated model invocation without cross-condition reuse.

Model configuration and call semantics. The declared generation model is `gpt-5-mini` as recorded in `execution/model_configuration.json`. Execution parameters recorded in that file include `max_output_tokens = 1500` and `maximum_attempts_per_call = 1`. Provider field `'openai_responses'` documents the execution provenance; no retrieval augmentation was used (`retrieval_augmented_generation = false` in the preregistration). The single-call, single-candidate policy is consistent with the preregistered budget (`model_calls_used`  $\leq$  `maximum_model_calls`) and with the planned `shared_candidate` estimand.

Deterministic validator: normalization, constraints, and scoring rules. The `deterministic_netops_validator_v5` implements the preregistered `full_intent_constraint_validation` scorer. Validation is applied in two phases recorded in scoring JSONs: `validation_before` (candidate normalization and difficulty/context diagnostics) and `validation_after` (post-orchestration final state simulation and constraint checks). Concrete checks include: - Syntax and canonical normalization: compute `normalized_response` and canonical ordering when semantically reorderable commands exist; `normalized_response` is archived in `execution/scoring/*.json`. -

Required command inclusion: verify that every entry in the task payload’s `required_sequence` is present in the candidate (field-level coverage) yielding `required_command_count` and `parsed_command_count`. - Ordering and dependency checks: enforce patterns such as make-before-break (up-link switchover), `shutdown_modify_restore` (MTU/VLAN changes requiring admin down), and group constraints (`minimum_active_in_groups`, `final_group_constraints`). Each pattern maps to validator rules that set `violation_codes` and `violation_count`. - Final state simulation: apply the candidate commands to a deterministic emulator model of device states and check that the resultant `final_state` matches expected `final_state` for the fields constrained by the task, producing `validation_after.valid` boolean recorded per episode.

Scoring semantics and success definition. Per preregistration, a task episode is scored success (binary 1) iff `validation_after.valid == true` and `scoring_rule == "full_intent_constraint_validation"`. The scoring JSONs include `score_reason_code` (e.g., `VALID_CONFIGURATION`) and `repair_applied` flag. All per-episode scoring artifacts (`execution/scoring/task-000***-*.json`) are the canonical record of the validator decision and include `raw_response_sha256`, `shared_initial_candidate_sha256`, and `validator_trace` with parsed counts and normalized configurations.

Repair policy and instrumentation. The preregistered `transformation_scope` allowed a single repair iteration per task (`maximum_repair_calls_per_task = 1`). The orchestration records `repair_provider_trace_path` and `repair_provider_trace_sha256` when repair is requested and applied. In this execution, aggregated `repair_applied` count = 0 across `execution/scoring/*.json` (`repair_applied == false` for every episode), therefore the repair loop remained uninvoked; this observation is archived and referenced in analysis artifacts.

Contamination checks, missingness, and treatment rules. The preregistered `contamination_plan` included (i) a simulator fidelity suite that compares emulator outputs to held-out public device examples and flags tasks with >5% disagreement, (ii) grammar coverage/unit tests for vendor CLI constructs, and (iii) execution isolation (explicit model context resets). `Analysis/contamination_summary.csv` is empty for this run (no flags). Missingness rules allowed two retries on transient provider failures and specified exclusion only when both pipeline runs were lost for a pair; the execution manifest reports `failed_episode_count = 0` and `missingness_summary` indicates `complete_pairs = 120`.

Analysis executor and inferential procedures. Analysis followed `paired_binary_analysis_v1` exactly as preregistered. The executor computed the  $2 \times 2$  paired contingency table ( $n_{11}, n_{10}, n_{01}, n_{00}$ ), the paired difference in success rates ( $A - B$ ), bootstrap percentile 95% CI using `bootstrap_resamples = 10000` and `bootstrap_seed = 1729` (both recorded in `analysis/analysis_log.jsonl`), and an exact McNemar two-sided test for the paired binary outcome. Analysis artifacts (`analysis/paired_contingency_table.csv`, `anal-`

ysis/condition\_summary.csv, analysis/analysis\_log.jsonl) and deterministic\_reconciliation.json provide reproducible inputs and outputs for independent recomputation.

Reproducibility and artifact references. Every described artifact is archived in the execution bundle and referenced in the execution manifest and analysis artifacts. Important file paths used by reviewers/readers: execution/execution\_manifest.json (execution accounting and preregistration\_sha256), execution/model\_configuration.json (gpt-5-mini declaration), execution/task\_manifest.jsonl (task payloads and difficulty fields), execution/responses/\* (shared and per-condition textual candidates), execution/scoring/\*.json (per-episode validator traces and scoring decisions), analysis/paired\_contingency\_table.csv and analysis/condition\_summary.csv (primary outcomes), and analysis/analysis\_log.jsonl (bootstrap parameters). These artifacts collectively document the operational semantics: single generation per pair → identical candidate delivered to both arms → deterministic normalization and constraint validation → binary success decision archived per episode.

Protocol limitations relevant to methodology. The shared\_initial\_candidate semantics were a preregistered design choice and implemented deterministically; it removes generator sampling variance and the operational effect of independent repairs. The methodology above therefore describes an exactly specified validator-acceptance parity measurement: it assesses downstream orchestration and validator agreement on identical textual candidates rather than independent generator robustness. Practical follow-ups to test independent generation and repair require concrete protocol changes (remove shared\_candidate, increase model\_calls\_used to generate independent candidates per arm, enable repair calls) that are specified in the manuscript’s Discussion and documented in the preregistration as amendable, auditable modifications.

#### IV. RESULTS

Execution accounting and primary numeric outcomes. The execution manifest records planned\_episode\_count = 240, completed\_episode\_count = 240, failed\_episode\_count = 0, model\_calls\_used = 120 (one shared LLM call per paired task), and maximum\_model\_calls = 240. Deterministic reconciliation (analysis/deterministic\_reconciliation.json) confirms complete\_pair\_count = 120 and provider audit counts: hosted\_model\_call\_event\_count = 120, completed\_hosted\_model\_call\_event\_count = 120, failed\_hosted\_model\_call\_event\_count = 0, cache\_miss\_count = 120, cache\_hit\_count = 0, and cross\_condition\_cache\_key\_reuse\_observed = false. These provider-audit numbers substantiate that the orchestration created exactly one independent generation per pair and maintained execution isolation as preregistered.

Primary contingency and statistical summaries. The analysis contingency (analysis/paired\_contingency\_table.csv) is  $n_{11} = 120$ ,  $n_{10} = 0$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ . Marginal success rates are baseline =  $120/120 = 1.0$  and guarded =  $120/120 = 1.0$ ; the observed paired difference  $A - B = 0.00$ . The

paired bootstrap percentile 95% CI was computed with bootstrap\_resamples = 10,000 and bootstrap\_seed = 1729 (analysis/analysis\_log.jsonl) and returned [0.00, 0.00]. The exact McNemar two-sided test yields  $p = 1.0$  (test applied to the observed discordant counts  $n_{10}$  and  $n_{01}$ ). These statistics are deterministic given the degenerate contingency table.

Repair and secondary outcomes. Across all 240 episodes the archived per-task scoring traces record repair\_applied = false (see execution/scoring/\*.json). Aggregation yields repair\_applied count = 0; consequently preregistered secondary estimands that quantify repair coverage and mean repair iterations are unobserved in this execution. The absence of repair activity is present in every representative scoring JSON (for example, execution/scoring/task-000001-baseline.json records repair\_applied: false and validator\_trace.validation\_after.valid: true), and is consistent across the artifact set.

Representative per-task diagnostics. Representative scoring artifacts show that shared\_initial\_candidate matched the normalized\_reference\_answer modulo command ordering in several cases and that validators parsed and validated required commands: example excerpts from archived scoring files illustrate typical complexity and validator checks— • task-000001: parsed\_command\_count = 4, required\_command\_count = 4, validation\_after.valid = true; normalized\_configuration and final\_state fields show the expected interface admin/mtu/vlan values and no violations (execution/scoring/task-000001-\*.json). • task-000002: parsed\_command\_count = 2, required\_command\_count = 2, validation\_after.valid = true; difficulty flagged as level = hard with pattern = make\_before\_break\_uplink\_switchover. • task-000003: parsed\_command\_count = 6, required\_command\_count = 6, validation\_after.valid = true; difficulty = hard with pattern = paired\_link\_atomic\_mtu\_change. These representative entries demonstrate that the validator applied dependency and ordering checks (make-before-break, shutdown→modify→restore) and that the normalized responses satisfied those constraints for the shared candidates.

Contamination and missingness diagnostics. The preregistered contamination checks produced no flags: analysis/contamination\_summary.csv is empty. Missingness diagnostics (analysis/missingness\_summary.csv) report complete\_pairs = 120 and zero for baseline\_only\_observed\_pairs, guarded\_only\_observed\_pairs, both\_missing\_pairs, and failed episodes. Provider audit and execution logs show no retries or infrastructure failures beyond the planned single generation per pair; execution/execution\_manifest.json and analysis/deterministic\_reconciliation.json contain the machine-readable accounting.

Interpretation constrained to measured estimand. Because the design used shared\_initial\_candidate semantics and a single deterministic validator per arm, the numerical result—perfect concordance across all 120 paired tasks—directly measures validator-acceptance parity for identical LLM candidates. The degenerate contingency produces a point mass estimate (difference = 0.00) and a degenerate bootstrap CI and

p-value; these are accurate descriptions of the observed data. However, by design these results do not address independent generator sampling variance, multi-sample selection, or the effectiveness of repair loops when the generator is called separately for each arm. Those secondary dimensions remained unexercised here and are therefore unobserved in the artifacts.

## V. DISCUSSION

What the run measures — and what it does not. Because `shared_initial_candidate` semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance ( $n_{11} = 120$ ) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Evidence-to-claim mapping (compact table). Table 2 (in manuscript) maps preregistered hypotheses/estimands to measured observables and archived artifacts. Example mappings: H1 (paired difference  $A - B$ )  $\rightarrow$  measured estimand = validator-acceptance parity under `shared_initial_candidate`; supporting artifacts: `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv` ( $n_{11}=120, n_{10}=0, n_{01}=0, n_{00}=0$ ); verdict: tested but restricted. H2 (agentic repair coverage advantage)  $\rightarrow$  observed `repair_applied` count = 0 in `execution/scoring/*.json`; verdict: unobserved in this execution.

Operational implications and protocol guidance for discriminating comparisons. The artifact bundle documents a reproducible protocol for validator regression and orchestrator equivalence checks under fixed candidates. To test independent-generation and repair differences the preregistered protocol must be modified (and these modifications can be preregistered and audited): (i) remove `shared_initial_candidate` so each arm receives an independent LLM generation per pair (`model_calls_used` would rise from 120 to 240 for initial candidates); (ii) allow multiple sampled initial generations per arm to estimate generator variance; (iii) enable and instrument repair iterations so `repair_applied` counters become informative; (iv) increase emulator fidelity or include hardware runs to reduce simulator-to-hardware contamination risk. Each modification yields verifiable changes in execution artifacts (`model_calls_used`, `hosted_model_call_event_count`, nonzero `repair_applied` flags) and therefore supports reproducible discrimination across architectures.

Threats to validity (concise). Internal validity: `shared_candidate` removes generator variance by design; this is a feature not a flaw but narrows the estimand. Construct validity: deterministic validator acceptance operationalizes a conservative correctness definition but omits

human-facing properties (explainability, maintainability). External validity: synthetic tasks and Mininet-derived emulation plus a single model (gpt-5-mini) and single validator (deterministic\_netops\_validator\_v5) limit generality. Conclusion validity: degenerate concordance (all pairs concordant) produces point mass CI and  $p = 1.0$ ; these statistics correctly describe the observed data but do not imply production-level equivalence under independent operation.

Reproducibility and audit artifacts. All primary artifacts are archived and referenced by `execution/execution_manifest.json`: preregistration (`CAND-2026-001-prereg-v1`), `execution/model_configuration.json` (declared `model_version = gpt-5-mini`), `execution/task_manifest.jsonl`, `execution/responses/*`, `execution/scoring/*.json`, `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, `analysis/analysis_log.jsonl`, and `analysis/deterministic_reconciliation.json`. Per-task validator traces (`execution/scoring/task-000XXX-*.json`) are the canonical records for each scored decision. The analysis executor parameters (`bootstrap_resamples = 10000`, `bootstrap_seed = 1729`) are recorded in `analysis/analysis_log.jsonl` and enable deterministic recomputation of the reported CI.

## VI. LIMITATIONS

- Preregistered `shared_initial_candidate` semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (deterministic\_netops\_validator\_v5) were used; results do not imply multi-model or multi-validator generality.
- No repairs were applied in this run (`repair_applied = false` in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

## VII. CONCLUSION

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate $\rightarrow$ validate $\rightarrow$ repair NetOps pipelines under `shared_initial_candidate` semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair ( $n_{11} = 120, n_{10} = n_{01} = n_{00} = 0$ ), yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar  $p = 1.0$ ). No repairs were applied (`repair_applied`

= false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

#### DISCLOSURE STATEMENT

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration\_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared\_model\_version in execution/model\_configuration.json); provider record 'openai\_responses'. The hosted adapter family was hosted\_netops\_gvr\_v1. Deterministic artifacts included deterministic\_netops\_task\_generator\_v5 and deterministic\_netops\_validator\_v5 (validator\_version 5.0). The run deliberately used the preregistered shared\_initial\_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model\_calls\_used = 120 (one shared generation per pair) while planned episodes were 240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by paired\_binary\_analysis\_v1. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial master prompt: provenance/master\_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd1582bb39b15ba). The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

#### REFERENCES

- [1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909
- [2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881
- [3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391
- [4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993
- [5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318